МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

(Самарский университет)

ОТЧЕТ ПО

ЛАБОРАТОРНОЙ РАБОТЕ №3

«Работа с исключениям и интерфейсами в наборе классов табулированной функции»

По курсу

Объектно-ориентированное программирование

Работу выполнила:

студент группы 6201-120303D,

Нагуманова В. Э.

Самара 2025

Содержание

Задание №1.....3

Задание №2.....4

Задание №3.....5

Задание №4.....8

Задание №5.....12

Задание №6.....15

Задание №7.....16

Задание №1.

Перед началом работы с кодом, я, следуя заданию, изучила следующие классы исключений, входящие в API Java:

• java.lang.Exception – базовый класс для всех исключений, которые могут возникнуть во время выполнения программы;

• java.lang.IndexOutOfBoundsException – для проверки исключений, когда индекс выходит за границы диапазона;

• java.lang.ArrayIndexOutOfBoundsException – для проверки доступа к элементу массива по недопустимому индексу;

• java.lang.IllegalArgumentException – неверный аргумент метода;

• java.lang.IllegalStateException – неподходящее состояние объекта.

Задание №2.

Для выполнения этого задания в пакете functions я создала два класса исключений:

- FunctionPointIndexOutOfBoundsExeption – исключение выхода за границы набора точек при обращении к ним по номеру, наследует от класса IndexOutOfBoundsException;

- InappropriateFunctionPointException – исключение, выбрасываемое при попытке добавления или изменения точки функции несоответствующим образом, наследует от класса Exception.

```
package functions;

//объявление класса исключений
public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {  no usages

    //конструктор по умолчанию
    public FunctionPointIndexOutOfBoundsException() {  no usages
        super(); //вызов конструктора родительского класса
    }

    //конструктор с сообщением
    public FunctionPointIndexOutOfBoundsException(String message) {  no usages
        super(message); //передаем сообщение в родительский класс
    }
}
```

Рис. 1

```java
package functions;

public class InappropriateFunctionPointException extends Exception {  no usages
    //конструктор по умолчанию
    public InappropriateFunctionPointException(){  no usages
        //вызов конструктора родителя
        super();
    }
    //конструктор с сообщением
    public InappropriateFunctionPointException(String message){  no usages
        // передаем сообщение родителю
        super(message);
    }
}
```

Рис. 2

Задание №3.

Для этого задания в уже разработанный класс TabulatedFunction вносим изменения в конструкторы:

- оба конструктора класса должны выбрасывать исключение IllegalArgumentException, если левая граница области определения больше или равна правой, а также если предлагаемое количество точек меньше двух;

и в методы:

- методы getPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY(), deletePoint() должны выбрасывать исключение FunctionPointIndexOutOfBoundsException;

- методы setPoint() и setPointX() должны выбрасывать исключение InappropriateFunctionPointException в том случае, если координата x задаваемой точки лежит вне интервала, определяемого значениями соседних точек табулированной функции. Метод addPoint() также должен выбрасывать исключение InappropriateFunctionPointException, если в наборе точек функции есть точка, абсцисса которой совпадает с абсциссой добавляемой точки. Это обеспечит сохранение упорядоченности точек функции;

- метод deletePoint() должен выбрасывать исключение IllegalStateException, если на момент удаления точки количество точек в наборе менее трех. Это обеспечит невозможность получения функции с некорректным количеством точек.

```java
//создаёт объект табулированной функции по заданным левой и правой границам области определения, а также количеству точек для табулирования
public TabulatedFunction(double leftX, double rightX, int pointsCount) throws IllegalArgumentException{  no usages

    if(leftX >= rightX) throw new IllegalArgumentException("Левая граница больше правой.");
    if(pointsCount < 2) throw new IllegalArgumentException("Количество точек меьше двух.");

    this.pointsCount = pointsCount;
    points = new FunctionPoint[pointsCount];
    double step = Math.abs(rightX - leftX) / (pointsCount - 1); //шаг между точками
    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint( x: leftX + i * step,  y: 0);
    }
}

//создает объект табулированной функции по заданным левой и правой границам области определения, а также значениям функции
public TabulatedFunction(double leftX, double rightX, double[] values) throws IllegalArgumentException {  no usages

    if(leftX >= rightX) throw new IllegalArgumentException("Левая граница должна быть меньше правой.");
    if(values.length < 2) throw new IllegalArgumentException("Количество точек меньше двух.");

    int pointsCount = values.length;
    this.pointsCount = pointsCount;
    points = new FunctionPoint[pointsCount];
    double step = Math.abs(rightX - leftX) / (pointsCount - 1); //интервал (шаг между точками)
    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint( x: leftX + i * step, values[i]);
    }
}
```

Рис. 3

```java
//создаёт объект табулированной функции по заданным левой и правой границам области определения, а также количеству точек для табулирования
public TabulatedFunction(double leftX, double rightX, int pointsCount) throws IllegalArgumentException{  no usages

    if(leftX >= rightX) throw new IllegalArgumentException("Левая граница больше правой.");
    if(pointsCount < 2) throw new IllegalArgumentException("Количество точек меьше двух.");

    this.pointsCount = pointsCount;
    points = new FunctionPoint[pointsCount];
    double step = Math.abs(rightX - leftX) / (pointsCount - 1); //шаг между точками
    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint( x: leftX + i * step,  y: 0);
    }
}

//создает объект табулированной функции по заданным левой и правой границам области определения, а также значениям функции
public TabulatedFunction(double leftX, double rightX, double[] values) throws IllegalArgumentException {  no usages

    if(leftX >= rightX) throw new IllegalArgumentException("Левая граница должна быть меньше правой.");
    if(values.length < 2) throw new IllegalArgumentException("Количество точек меньше двух.");

    int pointsCount = values.length;
    this.pointsCount = pointsCount;
    points = new FunctionPoint[pointsCount];
    double step = Math.abs(rightX - leftX) / (pointsCount - 1); //интервал (шаг между точками)
    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint( x: leftX + i * step, values[i]);
    }
}
```

Рис. 4

```
}

//возвращает копию точки по указанному индексу
public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException{  no usages
    if(index<0 || index >=pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");

    return new FunctionPoint(points[index]);
}

//заменяет указанную точку табулированной функции на переданную
public void setPoint(int index, FunctionPoint point) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException{  no usages
    if(index<0 || index>= pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");

    if ( ( index != 0 && point.getX() <= points[index - 1].getX()) || (index != pointsCount-1 && point.getX() >= points[index+1].getX())) //текущий
        throw new InappropriateFunctionPointException("Неверное значение X. Точка должна входить в интервал.");

    points[index]= new FunctionPoint(point);
}
```

Рис. 5

```
    //возвращает значение абсциссы точки с указанным номером
    public double getPointX(int index) throws FunctionPointIndexOutOfBoundsException{  no usages
        if(index<0 || index>= pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");
        return points[index].getX();
    }

    //измененяет значения x
    public void setPointX(int index, double x) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException{  no usages
        if (index < 0 || index >= pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");
        if ((index != 0 && x <= points[index - 1].getX()) || (index != pointsCount-1 &&  x >= points[index + 1].getX()))
            throw new InappropriateFunctionPointException("Неверное значение X. Точка должна входить в интервал");

        points[index].setX(x);
    }

    //возвращает координаты y
    public double getPointY (int index) throws FunctionPointIndexOutOfBoundsException {  no usages
        if (index < 0 || index >= pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");
        return points[index].getY();
    }

    //изменяет значения y
    public void setPointY(int index, double y)throws FunctionPointIndexOutOfBoundsException{  no usages
        if (index < 0 || index >= pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");

        points[index].setY(y);
    }
```

Рис. 6



```
//удаление точки по указанному индексу
public void deletePoint(int index) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {  no usages
    if(index < 0 || index >= pointsCount) throw new FunctionPointIndexOutOfBoundsException("Точки не существует, недопустимое значение индекса.");
    if(pointsCount<3) throw new IllegalArgumentException("Недосточное количество точек.");

    System.arraycopy(points,  srcPos: index + 1, points, index,  length: pointsCount - index - 1);
    points[--pointsCount] = null;
}

//добавление новой точки в массив с сохранением порядка по X
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  no usages
    int index= 0;

    double newX = point.getX();
    //находим позицию для вставки точки
    while (index < pointsCount && newX > points[index].getX())
        index++;

    if( index < pointsCount && Math.abs(points[index].getX() - newX) < 1e-10) throw new InappropriateFunctionPointException("Точка  с таким значением X уже существет.");

    //проверка для увелечение массива
    if (pointsCount >= points.length) {
        FunctionPoint[] newPoints = new FunctionPoint[points.length * 2];
        System.arraycopy(points,  srcPos: 0, newPoints,  destPos: 0, pointsCount);
        points = newPoints;
    }

    if (index < pointsCount)
        System.arraycopy(points, index, points,  destPos: index + 1,  length: pointsCount - index);

    points[index] = new FunctionPoint(point);
    pointsCount++;

}
```

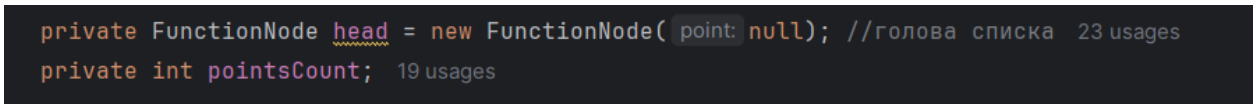
Рис. 7

Благодаря внедрению исключений, код сообщает об ошибках.

Задание №4.

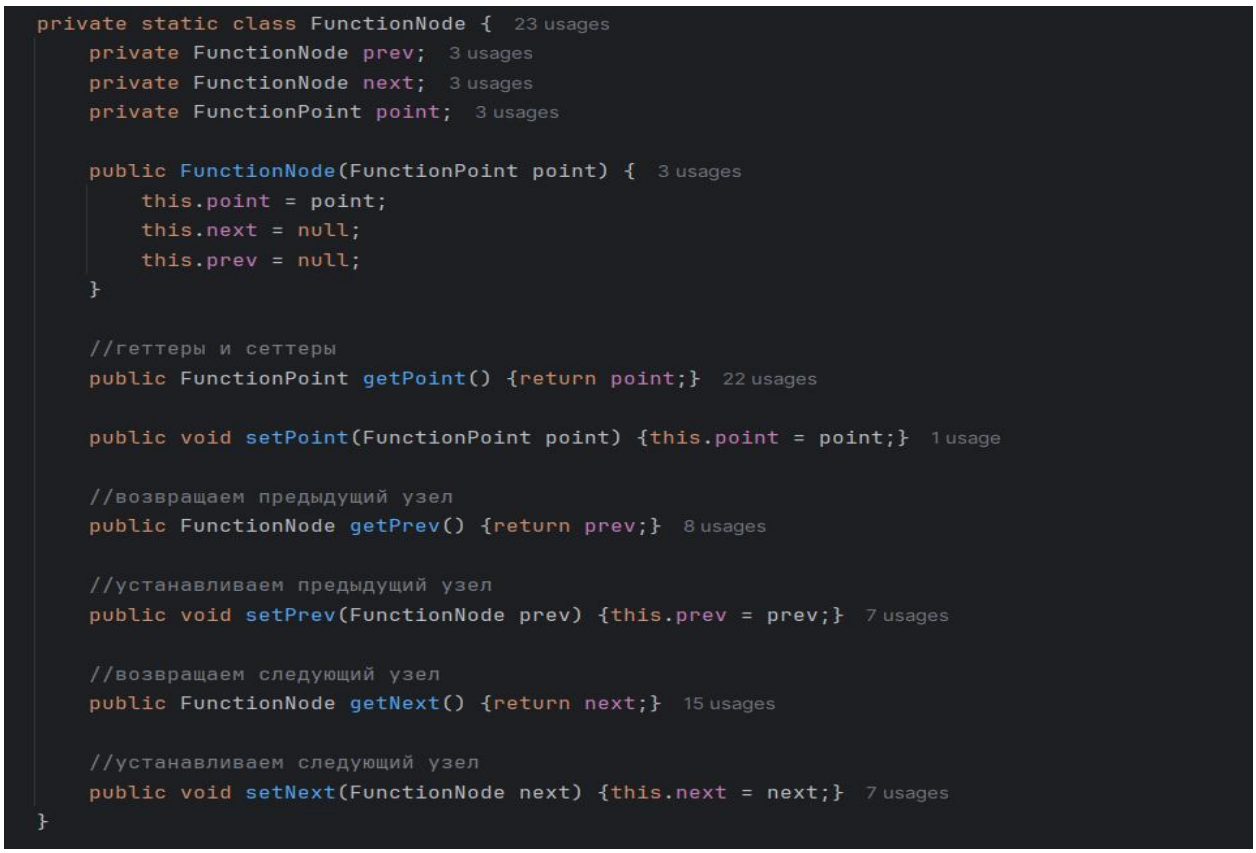
Для выполнения этого задания в пакете functions я создала класс LinkedListTabulatedFunction для работы с табулированной функцией, использующий для хранения точек двусвязный циклический список. Класс имеет два поля: начало списка, т.е. голова и количество точек.

Также был описан внутренний класс FunctionNode, узел списка, содержащий ссылки на следующий и предыдущий узлы, а также саму точку. Для работы с этими данными реализованы геттеры и сеттеры.



```
private FunctionNode head = new FunctionNode( point: null); //голова списка  23 usages
private int pointsCount;  19 usages
```

Рис. 8



```
private static class FunctionNode {  23 usages
    private FunctionNode prev;  3 usages
    private FunctionNode next;  3 usages
    private FunctionPoint point;  3 usages

    public FunctionNode(FunctionPoint point) {  3 usages
        this.point = point;
        this.next = null;
        this.prev = null;
    }

    //геттеры и сеттеры
    public FunctionPoint getPoint() {return point;}  22 usages

    public void setPoint(FunctionPoint point) {this.point = point;}  1 usage

    //возвращаем предыдущий узел
    public FunctionNode getPrev() {return prev;}  8 usages

    //устанавливаем предыдущий узел
    public void setPrev(FunctionNode prev) {this.prev = prev;}  7 usages

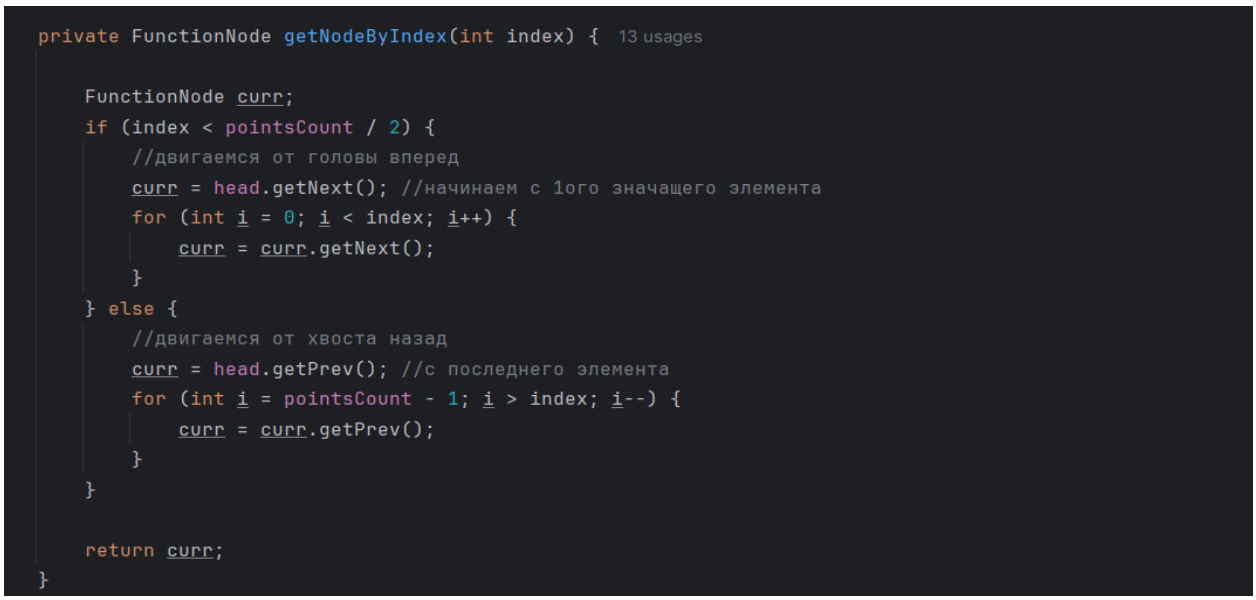
    //возвращаем следующий узел
    public FunctionNode getNext() {return next;}  15 usages

    //устанавливаем следующий узел
    public void setNext(FunctionNode next) {this.next = next;}  7 usages
}
```

Рис. 9

После этого мною были описаны методы:

- Метод getNodeByIndex(int index) находит и возвращает узел списка по его порядковому номеру. Для ускорения работы метод определяет, ближе ли нужный индекс к началу или концу списка, и начинает поиск с соответствующей стороны;



```
private FunctionNode getNodeByIndex(int index) {  13 usages

    FunctionNode curr;
    if (index < pointsCount / 2) {
        //двигаемся от головы вперед
        curr = head.getNext(); //начинаем с 1ого значащего элемента
        for (int i = 0; i < index; i++) {
            curr = curr.getNext();
        }
    } else {
        //двигаемся от хвоста назад
        curr = head.getPrev(); //с последнего элемента
        for (int i = pointsCount - 1; i > index; i--) {
            curr = curr.getPrev();
        }
    }

    return curr;
}
```

Рис. 10

- метод addNodeToTail() добавляет новый элемент в конец списка и возвращает на него ссылку. За счет обновления связей между узлами, новый узел "встраивается" между последним существующим элементом и первым, сохраняя кольцевую структуру списка;

```
//добвление в конец
private FunctionNode addNodeToTail(FunctionPoint point) {  3 usages

    FunctionNode tail = head.getPrev();
    FunctionNode newNode = new FunctionNode(point);

    tail.setNext(newNode);
    newNode.setPrev(tail);
    newNode.setNext(head);
    head.setPrev(newNode);

    pointsCount++;

    return newNode;
}
```

Рис. 11

- метод addNodeByIndex вставляет новый узел в любое место списка по указанному индексу и возвращает ссылку на этот узел. Для добавления используется тот же принцип, что и до этого;

```
private FunctionNode addNodeByIndex(int index, FunctionPoint point) {  1 usage
    FunctionNode newNode = new FunctionNode(point);

    if (index == pointsCount)
        return addNodeToTail(point); // добавляем в конец

    FunctionNode currNode = getNodeByIndex(index);

    newNode.setPrev(currNode.getPrev());
    newNode.setNext(currNode);
    currNode.getPrev().setNext(newNode);
    currNode.setPrev(newNode);

    pointsCount++;

    return newNode;
}
```

Рис. 12

- метод deleteNodeByIndex(int index) удаляет узел из связного списка по заданной позиции, возвращая ссылку на удаленный объект. Как и в методах добавления, реализация основана на обновлении связей между узлами, но в данном случае целевой элемент "извлекается" из цепочки путем перенаправления ссылок соседних элементов друг на друга;

```
private FunctionNode deleteNodeByIndex(int index) {  1 usage

    FunctionNode delNode = getNodeByIndex(index);

    delNode.getNext().setPrev(delNode.getPrev());
    delNode.getPrev().setNext(delNode.getNext());

    pointsCount--;

    return delNode;
}
```

Рис.13

Задание №5.

В этом задании я создала конструкторы и методы для двусвязного списка, как и в TabulatedFunction.

```
    //создание табулированной функции
    public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {  no usages
        if (leftX >= rightX)
            throw new IllegalArgumentException("Левая граница больше правой");
        if (pointsCount < 2)
            throw new IllegalArgumentException("Количество точек меньше двух");

        head.setPrev(head);
        head.setNext(head);
        this.pointsCount = 0;

        double step = Math.abs(rightX - leftX) / (pointsCount - 1);
        for (int i = 0; i < pointsCount; i++) {
            FunctionPoint point = new FunctionPoint( x: leftX + i * step,  y: 0);
            addNodeToTail(point);
        }
    }

    //табулированная функция с заданными значениями по оси ординат
    public LinkedListTabulatedFunction(double leftX, double rightX, double[] value) {  no usages
        if (leftX >= rightX) {
            throw new IllegalArgumentException("Левая граница больше правой");
        }
        if (value.length < 2) {
            throw new IllegalArgumentException("Количество точек меньше двух");
        }

        head.setPrev(head);
        head.setNext(head);
        this.pointsCount = 0;

        double step = Math.abs(rightX - leftX) / (value.length - 1);
        for (int i = 0; i < value.length; i++) {
            FunctionPoint point = new FunctionPoint( x: leftX + i * step, value[i]);
            addNodeToTail(point);
        }
    }
```

Рис. 14

Границы области определения определяются по первому и последнему элементам списка, которые располагаются соответственно после головного элемента и до него.

```
//возвращение левой границы
public double getLeftDomainBorder() {return head.getNext().getPoint().getX();}  1 usage

//возвращение правой границы
public double getRightDomainBorder() {return head.getPrev().getPoint().getX();}  1 usage
```

Рис. 15

Далее в методах getFunctionalValue, addPoint и вспомогательном методе вывода outFunction я изменила принцип доступа к элементам.

В методе getFunctionalValue вместо обращения к узлам списка по индексу, требующего последовательного прохода от начала до нужного элемента на каждой итерации, я реализовала прямой обход списка. В методах addPoint и outFunction так же, вместо обращения по индексу теперь используется прямое обращение к элементам.

```
//возвращение y, если точка лежит в области определения функции
public double getFunctionValue(double x) {  no usages
    double leftX = getLeftDomainBorder();
    double rightX = getRightDomainBorder();

    if (x >= leftX && x <= rightX) {

        FunctionNode curr = head.getNext();
        while (curr != head && curr.getNext() != head) {
            double x1 = curr.getPoint().getX();
            double x2 = curr.getNext().getPoint().getX();

            // совпадает ли x
            if (Math.abs(x - x1) < 1e-10)
                return curr.getPoint().getY();
            if (Math.abs(x - x2) < 1e-10)
                return curr.getNext().getPoint().getY();
            // попадает ли x в интервал
            if (x > x1 && x < x2) {
                double y1 = curr.getPoint().getY();
                double y2 = curr.getNext().getPoint().getY();
                return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
            }
            // переходим к следующему
            curr = curr.getNext();
        }
    }
    return Double.NaN;
}
```

Рис. 16

```java
//возвращения количества точек
public int getPointsCount() {  no usages
    return pointsCount;
}

//возвращение точки
public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException {  no usages
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Не существует точки");
    double x = getNodeByIndex(index).getPoint().getX();
    double y = getNodeByIndex(index).getPoint().getY();
    return new FunctionPoint(x, y);
}

//изменение указанной точки на заданную
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {  no usages
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Не существует точки");
    if ((index != 0 && point.getX() <= getNodeByIndex(index - 1).getPoint().getX()) ||
            (index != pointsCount - 1 && point.getX() >= getNodeByIndex(index + 1).getPoint().getX()))
        throw new InappropriateFunctionPointException("Неверный X");

    getNodeByIndex(index).setPoint(new FunctionPoint(point));

}

//возвращение координаты x
public double getPointX(int index) throws FunctionPointIndexOutOfBoundsException {  no usages
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Не существует точки");
    return getNodeByIndex(index).getPoint().getX();
}
```

Рис. 17

```java
//изменение значения x
public void setPointX(int index, double x) throws InappropriateFunctionPointException {  no usages
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Не существует точки");
    if ((index != 0 && x <= getNodeByIndex(index - 1).getPoint().getX()) ||
            (index != pointsCount - 1 && x >= getNodeByIndex(index + 1).getPoint().getX()))
        throw new InappropriateFunctionPointException("Неверный X");

    getNodeByIndex(index).getPoint().setX(x);
}

//возвращения координаты y
public double getPointY(int index) throws FunctionPointIndexOutOfBoundsException {  no usages
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Не существует точки");
    return getNodeByIndex(index).getPoint().getY();
}

//изменения значения y
public void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException {  no usages
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Не существует точки");
    getNodeByIndex(index).getPoint().setY(y);
}
```

Рис. 18

```java
//добавление точки
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  no usages
    int index = 0;

    double newX = point.getX();

    // находим позицию для вставки точки
    FunctionNode curr = head.getNext();
    while (curr != head && newX > curr.getPoint().getX()) {
        curr = curr.getNext();
        index++;
    }

    if (curr != head && Math.abs(curr.getPoint().getX() - newX) < 1e-10)
        throw new InappropriateFunctionPointException("Точка с таким X уже существует");

    addNodeByIndex(index, point);

}

// вывод массива точек
public void outFunction() {  no usages
    FunctionNode curr = head.getNext();
    while (curr != head) {
        System.out.printf("(%.2f, %.2f) ", curr.getPoint().getX(), curr.getPoint().getY());
        curr = curr.getNext();
    }
    System.out.println();
}
```

Рис. 19

Задание №6.

Следуя заданию, я переименовала класс TabulatedFunction в класс ArrayTabulatedFunction и создала интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction.

Задание №7.

В этом задании я проверила все основные методы: создание функций, добавление и удаление точек, вычисление значений, специально передавала неправильные данные, чтобы убедиться, что исключения работают как нужно.

```
Функиця: y = 2x + 2
(-3,50, -5,00) (-1,38, -0,75) (0,75, 3,50) (2,88, 7,75) (5,00, 12,00)

Проверка удаления точки с индексом 2
(-3,50, -5,00) (-1,38, -0,75) (2,88, 7,75) (5,00, 12,00)

Проверка добавления точки (1, 4)
(-3,50, -5,00) (-1,38, -0,75) (1,00, 4,00) (2,88, 7,75) (5,00, 12,00)

Проверка установки точки (-2, -2) на позицию с индексом 1
(-3,50, -5,00) (-2,00, -2,00) (1,00, 4,00) (2,88, 7,75) (5,00, 12,00)

Проверка getFunctionValue
(-1,00, 0,00)
(0,00, 2,00)
(1,00, 4,00)
(2,00, 6,00)


Тестирование исключений

Некорректное создание (левая граница больше правой)
Error: Левая граница больше правой.

Обращение к точке с несуществующим индексом
Error: Не существует точки

Точка нарушает порядок возрастания
Error: Неверный X

Обращение к точке с отрицательным индексом
Error: Не существует точки

Добавление точки с уже существующим X
Error: Точка с таким X уже существует
Error: Недосточное количество точек.
```

Рис. 20